Time Complexity of the Euclidean GCD Algorithm

Setup

Each step of division can be written using quotient and remainder. Let

$$a = q_0 b + r_0, \qquad b = q_1 r_0 + r_1, \qquad r_0 = q_2 r_1 + r_2, \qquad \dots$$

so in general

$$r_{i-1} = q_{i+1} r_i + r_{i+1}, \qquad 0 \le r_{i+1} < r_i.$$

Here $r_{-1} = a$ and r_0 is the first remainder (b plays the role of r_{-1}'s divisor). Each r_i is exactly the value stored in the code's variable b after i passes through the loop.

Since $a, b < 2^n$, we have $r_0 < 2^n$ as well.

Claim: Remainders Shrink by Half Every Two Steps

Claim. $2.r_{i+2} < r_i$ for all i.

Proof. From the division above,

$$r_{i-1} = q_{i+1} r_i + r_{i+1}.$$

Case 1: $q_{i+1} \ge 2$. Then $r_{i-1} \ge 2r_i$, i.e. $r_i \le r_{i-1}/2$. Since remainders are decreasing, in particular $r_{i+1} \le r_i \le r_{i-1}/2$. (Reindexing this argument one step later gives $r_{i+2} < r_i/2$ directly, since the same halving applies to consecutive pairs.)

Case 2: $q_{i+1} = 1$. Then $r_{i+1} = r_{i-1} - r_i$. But since $r_i \le r_{i-1}$ was itself obtained the same way, one can check $r_i < r_{i-1}$ strictly, so

$$r_{i+1} = r_{i-1} - r_i < r_{i-1} - 0,$$

and more precisely, since $q_{i+1} = 1$ means $r_i > r_{i-1}/2$ (otherwise the quotient would be ≥ 2), we get

$$r_{i+1} = r_{i-1} - r_i < r_{i-1} - \frac{r_{i-1}}{2} = \frac{r_{i-1}}{2}.$$

In both cases, two consecutive divisions cut the remainder to less than half of what it was two steps earlier:

$$2.r_{i+2} < r_i.$$

Counting the Steps

Since $r_0 < 2^n$, applying the claim repeatedly gives

$$r_{2k} < \frac{2^n}{2^k}.$$

At $k = n$:

$$r_{2n} < \frac{2^n}{2^n} = 1,$$

so $r_{2n} = 0$, since remainders are nonnegative integers. This means the algorithm has already terminated (the loop condition `while(b != 0)` fails) at or before step $2n$.

Conclusion

The chain of divisions $a = q_0 b + r_0,\ b = q_1 r_0 + r_1, \ldots$ can have at most $2n$ terms before reaching remainder 0, since the remainder is cut in half every two divisions. Each division is $O(1)$ work, so the algorithm runs in

$$O(n)$$

time — linear in the bit-length n of the inputs.

Reference

- A. V. Aho, J. E. Hopcroft, and J. D. Ullman, *The Design and Analysis of Computer Algorithms*, Addison-Wesley, 1974.
- V. Shoup, *A Computational Introduction to Number Theory and Algebra*, Cambridge University Press, 2009.